BİL4010

Paralel programlama

5.Hafta

Paralel yazılım Mimarileri

İçerik

- Paralel Yazılımlar
  - Hafıza Paylaşımlı Programlar
  - Dağıtık Hafızalı Programlar
- Performans
  - Speedup ve Verimlilik
  - Amdahl Yasası
  - Ölçeklenebilirlik
- Parelel Program Tasarımı
  - Foster Yöntemi

Parelel Donanım ve Yazılımlar

- Verimli paralel programlar yazabilmek için paralel olarak çalışacak programın üzerinde çalışacağı bilgisayar donanımı ve işletim sistemi hakkında bilgi sahibi olunması gerekir.
- Bu bölümde paralel yazılım yöntemlerinden bahsedilecektir.

Parelel Yazılım

- Günlük hayatta paralel donanımlar PC'ler ve Sunucu bilgisayarlarda yaygın olarak kullanılmaktadır, ancak aynı durum paralel yazılımlar için geçerli mi?
- Paralel yazılımlar iki alt başlıkta incelenmektedir:
  - Hafıza Paylaşımlı ve Dağıtık Hafızalı sistemler
- Paylaşımlı hafızalarda çalışan programlar tek bir işlem (process) üzerinden çoklu iş parçacıkları (thread) kullanarak çalışırlar.
- Dağıtık sistemlerde ise görevleri (tasks) gerçekleştirmek için çoklu işlem (multiprocess) yapılarına ihtiyaç duyulmaktadır.

Hafıza Paylaşımlı Programlar

- Hafıza paylaşımlı programlarda değişkenler (variables) paylaşılabilir ve public veya özel (private) olabilir.
  - Paylaşılan (public) değişkenler herhangi bir iş parçacığı (thread) tarafından okunabilir ve yazılabilirken, özel (private) değişkenler normalde yalnızca bir iş parçacığı üzerinden erişilebilir.
  - İş parçacıkları (thread) arasındaki iletişim genelikle paylaşılan değişkenler aracılığıyla yapılır.
  - Bu nedenle iş parçacıkları (thread) iletişim örtülü (implict) bir şekilde gerçekleştirilir.

Dinamik ve Statik Thread'ler

- Birçok yazılım geliştirme ortamı paylaşımlı hafıza programları için dinamik iş parçacıkları (threads) kullanır.
- Genellikle bu tür yapılar bir ana iş parçacığı ve buna bağlı işçi thread'lerden oluşmaktadır. Yeni bir görev (task) geldiğinde ana iş parçacığı bu görevi bir iş parçacığına atar.
- Statik Thread yapısında ise; tüm iş parçacıkları ana iş parçacığı tarafından gerekli ayarlamanın ardından çatallanır ve tüm iş tamamlanıncaya kadar iş parçacıkları çalıştırılır.
- Statik Thread'ler dinamiklere göre genellikle daha hızlı çalışırlar.

Nondeterminisim (Belirsizlik)

- İşlemcilerin asenkron olarak çalıştığı MIMD sistemlerinde muhtemel belirsizlikler söz konusudur.
- Belirli bir girdi farklı çıktılarla sonuçlanırsa hesaplama belirsizdir.
- Birden çok iş parçacığı bağımsız olarak çalışıyorsa, deyimleri tamamladıkları göreceli hız her çalıştırmada değişir ve dolayısıyla programın sonuçları her çalıştırmada farklı olabilir.
- Bu durum genelde büyük sorunlara yol açmaz.
- Bazı durumlarda ise eğer sistem kaynakları ortak kullanıyorsa bir iş parçacığının çıktısı diğer bir iş parçacığını engelleyebilir.

Örn: Nondeterminisim (Belirsizlik)

- Farz edelim ki 2 adet iş parçacığımız var; Thread 0 ve Thread 1
- Bu iş parçacıkları my_x adı verilen özel bir değişken hesaplıyorlar. Thread 0 için my_x = 7 ve Thread 1 için my_x = 19 olsun. Her iki iş parçacığı aşağıdaki kodu çalıştırdığında;

```
. . .
printf("Thread %d > my_val = %d\n", my_rank, my_x);
. . .
```

Örn: Nondeterminisim (Belirsizlik)

- Birinci çalıştırmada ekran çıktısı:

```
Thread 0 > my_val = 7
Thread 1 > my_val = 19
```

- İkinci çalıştırmada ise:

```
Thread 1 > my_val = 19
Thread 0 > my_val = 7
```

Şeklinde gerçekleşebilir.

Dağıtık Hafızalı Programlar

- Dağıtık hafızalı (bellek) programlarında, çekirdekler doğrudan sadece kendi özel hafızalarına erişebilir.
- Bu yapı için en yaygın kullanım yöntem Mesaj-geçici (message-passing) yöntemidir.
- Dağıtık hafızalı sistemler paylaşımlı bir hafıza üzerinde de kolaylıkla uygulanabilir.
- Programcı mantıksal olarak ortak belleği çeşitli iş parçacıkları için özel adres alanlarına bölebilir.
- Daha sonra bir kitaplık veya derleyici iş parçacıkları arasında gereken iletişimi uygulayabilir.

Mesaj-geçişi (Message-passing)

- Mesaj-geçişi API (Application Program Interface) en azından bir mesaj gönderim (send) ve alım (receive) fonksiyonu içermelidir.
- İşlemler tipik olarak birbirlerini rütlebeleri (rank) ile tanımlarlar. *p* adet işlem içeren dağıtık bir sistemde işlem rütbeleri (rank) *0, 1, ..., p-1* olarak tanımlanır.
- Mesaj-geçişi için en yaygın kullanılan API Message Passing Interface veya MPI'dir.

Örn: Mesaj-geçişi (Message-passing)

```
char message[100];
. . .
my_rank = Get_rank();
if (my_rank == 1) {
     sprintf(message, "Greetings from process 1");
     Send(message, MSG_CHAR, 100, 0);
}else if (my_rank == 0) {
     Receive(message, MSG_CHAR, 100, 1);
     printf("Process 0 > Received: %s\n", message);
}
```

Örn: Mesaj-geçişi (Message-passing)

- `Get rank()` fonksiyonu çağırılan işlemin rütbesini döndürür. Ardından işlemler rütbelerine göre dallanırlar.
- Rütbesi 1 olan işlem aşağıdaki fonksiyon

  `sprintf(message, "Greetings from process 1");` ile bir mesaj oluşturur ve

  `Send(message, MSG_CHAR, 100, 0);` fonksiyonu ile bunu işlem 0'a gönderir.
- İşlem 0 ise `Receive(message, MSG_CHAR, 100, 1);` fonksiyonu ile gelen mesajı alır.

  `message`: mesaj içeriği `MSG_CHAR`: mesaj türü `100`: mesaj uzunluğu

  `1`: mesajı gönderen işlem.

Tek taraflı iletişim (One-sided communication)

- Mesaj-geçişli sistemlerde, bir işlem mutlaka bir gönderme fonksiyonu çağırmalı ve gönderme fonksiyonu başka bir işlem tarafından bir alma fonksiyonu ile eşleştirilmelidir.
- Tek taraflı iletişim veya uzak hafıza erişimi (remote memory access) yapısında ise; tek bir işlem, yerel belleği başka bir işlemden veya uzak belleği çağıran bir işlemden gelen değerle güncelleyen bir fonksiyonu çağırır.
- İletişim her zaman tek yönlüdür.

Performans

- Paralel programlara ihtiyaç duyulmasının en büyük nedeni programın performansını (çalışma hızı, işlem kapasitesi vs) arttırmaktadır.
- Peki paralel programlar hangi performans kriterlere göre değerlendirilir?

Hızlanma (Speedup) ve Verimlilik (Efficiency)

- Farz edelim ki programımız p adet core bulunan bir sistem üzerinde tamamen paralel çalışıyor. Bu durumda programın çalışma süresi:

$$T_{parallel} = \frac{T_{serial}}{p}$$

ile ifade edilebilir. Bu, ideal durum olarak kabul edilir. Bu duruma doğrusal hızlanma (linear speedup) denmektedir.

- Fakat bu durum gerçekte pek mümkün olmayabilir ve speedup aşağıdaki formül ile hesaplanır.

$$S = \frac{T_{serial}}{T_{parallel}}$$

S(Speedup): Paralel programın hızlanma kriteri

Hızlanma (Speedup) ve Verimlilik (Efficiency)

- İdeal durumda (bu gerçek hayatta pek mümkün değil) $S = p$ olacaktır.
- Sistemin verimliliği (Efficiency - E) ise:

$$E = \frac{S}{p}$$ formülü ile hesaplanır.

- (*E: Verimlilik, S: Hızlanma, p: Çekirdek sayısı*) formülü biraz daha açarsak:

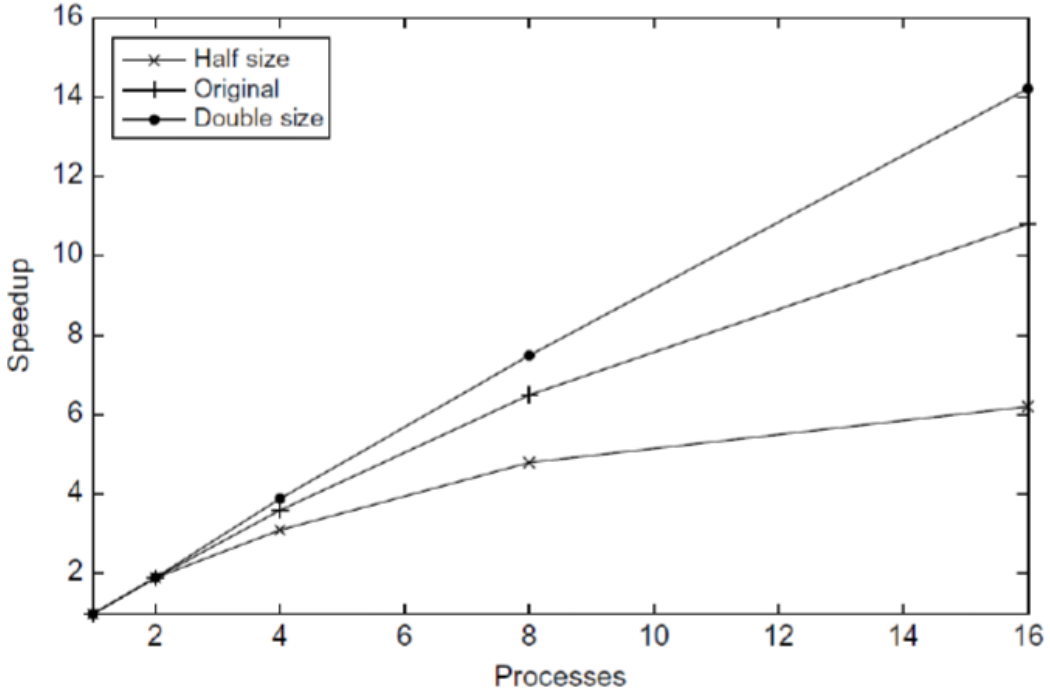
$$E = \frac{S}{p} = \frac{\left(\frac{T_{serial}}{T_{parallel}}\right)}{p} = \frac{T_{serial}}{p \,.\, T_{parallel}}$$

Hızlanma (Speedup) ve Verimlilik (Efficiency)

- $T_{parallel}$, E ve S değerleri p sayısına bağlıdır. Aynı zamanda problem uzayının büyüküğü bu değerlerin değişmesine neden olacaktır.

Table 2.5 Speedups and Efficiencies of a Parallel Program on Different Problem Sizes

	p	1	2	4	8	16
Half	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
Original	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
Double	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89



Paralelliştirme Maliyeti (Parallel Overhead)

- Gerçek hayatta herhangi bir program paralleştirilirken verinin core'lara dağıtılması ve tekrar birleştirilmesi gibi ek maliyetler ortaya çıkmakta ve bu da sistemin paralelleştirme süresinin üzerine bir yük olarak eklenmelidir.

$$T_{parallel} = \frac{T_{serial}}{p} + T_{overhead}$$

- İşlemlerin core'lara dağıtılıp toplanması süresi Seri hesaplama süresinden çok daha küçüktür.

$$T_{overhead} << T_{serial}$$

Amdahl Yasası

- 1960 yılında Gene Amdahl tarafından ortaya konulan yasa:
  - Bir programı tümüyle paralleştirmek mümkün değildir. Bu yüzden belirli bir seviyeden sonra core sayısını ne kadar arttırırsanız arttırın sistemi daha fazla hızlandıramazsınız.
- Örnek problem:
  - Bir programın %90 kadarının paralleştirilebildiği varsayıldığında, Tserial = 20 olsun. Ğaralleştirme işleminde overhead değerini ihmal edilerek toplam paralel programın çalışma süresi:

$$T_{parallel} = \frac{0.9\,.\,T_{serial}}{p} + 0.1\,.\,T_{serial} = \frac{18}{p} + 2$$

Amdahl Yasası

- S ise formülden: $S = \frac{T_{serial}}{T_{parallel}} = \frac{20}{18/p + 2}$

- Burada p (core sayısı) sonsuza giderken bu değer S = 10 olarak bulnur. Bu da speedup değerimizi S <= 10 olacağını ifade eder.
- Özetle Amdahl yasası sistemin seri kalan kısmının bizim Speedup değerimizin üst sınırını belirledini söylüyor. Bu örnek için bu değer:

$$S = \frac{1}{1 - \%T_{parallel}} = \frac{1}{1 - 0.9} = 10$$

Ölçeklenebilirlik

- Geliştirilen yazılım her geçen gün mevcut problemin boyutlarının artmasına rağmen çalışmaya devam edebiliyorsa ölçeklenebilirdir.
- Örneğin:
  - $T_{serial} = n$ micro saniye sürsün. Aynı zamanda problem boyutumuzun da n olduğunu varsayalım. $T_{paralel} = n/p+1$ paralel işlem süremiz olsun. Bu durumda:

$$E = \frac{S}{p} = \frac{T_{serial}}{p \,.\, T_{parallel}} = \frac{n}{p(\frac{n}{p}+1)} = \frac{n}{n+p}$$

Örn: Ölçeklenebilirlik

- Geliştirilen programın ölçeklenebilrliğini test etmek için problem boyumuzu k oranında işlemci sayımızı da x oranında arttıralım. Bu şartlar altında verimlilik:

$$E = \frac{n}{n+p} = \frac{xn}{xn+kp}$$

olarak hesaplanacaktır. Eğer x = k olursa bu denklem:

$$\frac{xn}{xn+kp} = \frac{kn}{k(n+p)} = \frac{n}{n+p}$$

olarak bulunur. Kısaca problem boyutunu işlemci sayısı ile aynı oranda büyütürsek paralel sistemin verimliliği **değişmeyecektir** ve **ölçeklenebilirlik** sağlanmış olacaktır.

Paralel Program Tasarımı

- Paralel donanım mimarileri belirli kriterler çerçevesinde tasarlanarak üretilebilmesine rağmen seri bir programı paralel bir programa çevirebilmek için kesin bir çözüm yolu bulunmamaktadır.

- Yine de Ian Foster'in geliştirmiş olduğu Paralel Program Tasarımı ve Mimarisi başlıklı eserde Paralel Programlama metodolojisi 4 adımda ele alınmıştır.

Paralel Programlama Metodolojisi

- Ian Foster'in yöntemi aşağıda verilen dört temel prensibe dayanmaktadır (Foster's methodology):
  1. Bölme (Partitioning)
  2. Haberleşme (Communication)
  3. Birleştirme (Aggregation)
  4. Planlama (Mapping)

Bölme (Partitioning)

- Gerçekleştirilecek hesaplama ve hesaplamada kullanılacak olan veriler küçük görevlere bölünmelidir. Buradaki odaklanma, paralel olarak yürütülebilecek görevlerin tanımlanması üzerine olmalıdır.
  - Divide and Conquer (Böl ve Yönet)

Haberleşme (Communication)

- Bir önceki adımda tanımlanan görevler arasında hangi iletişimin yapılması gerektiği belirlenmelidir:
  - Dağıtık bir sistem mi yoksa hafıza paylaşımlı bir sistem mi kullanılacak ?
  - Paralel sistem görevli arasında haberleşme nasıl olmalı?

Birleştirme (Aggregation)

- İlk adımda belirlenen görevleri ve iletişimleri daha büyük görevlerle birleştirmek.
- Örneğin, görev A, görev B yürütülmeden önce yürütülmeliyse, bunları tek bir bileşik göreve toplamak mantıklı olabilir.

Planlama (Mapping)

- Bir önceki adımda tanımlanan bileşik görevlerin süreçlere atanması işlemidir.
- Bu, iletişimin en aza indirgenmesi ve her işlemin kabaca aynı miktarda işle uğraşması için yapılmalıdır.

Teşekkürler.

Sorular...